SENAI FATESG

Sistema de Controle de Estoque SOS Estoque

Um problema dois paradigmas

Documentação técnica atualizada para a N2

Disciplina	Paradigmas de Programação
Curso	Engenharia de Software
Turma	Preencher antes da entrega
Grupo	Grupo 8
Integrantes	Guilherme Costa e Pedro Lucas Borges
Paradigmas	Orientado a objetos e procedural
Linguagens	Java e Python

Goiânia 2026

1 Resumo do projeto

O SOS-Estoque é um sistema de pequeno porte destinado ao controle de produtos e movimentações em pequenos comércios varejistas. A solução cadastra produtos, registra entradas e saídas, apresenta o inventário, identifica estoque crítico, calcula o valor financeiro armazenado, realiza buscas e mantém um histórico das movimentações.

O mesmo núcleo funcional foi implementado em duas abordagens. A primeira utiliza Java com orientação a objetos e interface web construída com Spring Boot e Thymeleaf. A segunda utiliza Python no estilo procedural, com execução pelo terminal, funções, listas e dicionários. As interfaces são diferentes, mas as regras e os resultados esperados são equivalentes.

2 Problema de mercado e público alvo

Pequenos comércios, como mercados de bairro, armarinhos e lojas locais, podem enfrentar dificuldades para acompanhar quantidades, entradas, saídas e necessidades de reposição. O controle manual aumenta o risco de divergências no saldo, falta de produtos e decisões baseadas em informações desatualizadas.

O público-alvo é formado por pequenos comerciantes, estoquistas e responsáveis operacionais de microempresas que precisam acompanhar o inventário diário com uma ferramenta simples e de baixo custo.

3 Objetivo e limites do escopo

O objetivo é oferecer um controle acadêmico de estoque que permita acompanhar produtos e movimentações e, ao mesmo tempo, demonstrar como paradigmas diferentes modificam a organização interna do software.

3.1 Funcionalidades incluídas

- Cadastro de produtos com código, nome, categoria, preço, quantidade e estoque mínimo.
- Registro de entradas e saídas com atualização imediata do saldo.
- Consulta e busca de produtos.
- Identificação de produtos em situação crítica.
- Cálculo do valor total armazenado.
- Histórico das movimentações realizadas durante a execução.

3.2 Funcionalidades fora do escopo

- Login, perfis de acesso e controle de usuários.
- Cadastro de fornecedores, faturamento, vendas e emissão de notas fiscais.
- Integração com sistemas externos e gerenciamento de filiais.
- Persistência em banco de dados ou arquivos após o encerramento.

4 Requisitos e regras de negócio

4.1 Requisitos funcionais

RF01 - Cadastro de produtos: Cadastrar produtos com código único, nome, categoria, preço unitário, quantidade atual e estoque mínimo.

RF02 - Movimentações: Registrar entradas e saídas e atualizar imediatamente o saldo do produto.

RF03 - Consulta de inventário: Exibir os produtos cadastrados e suas informações atualizadas.

RF04 - Estoque crítico: Listar produtos cuja quantidade seja menor ou igual ao estoque mínimo.

RF05 - Valor total: Calcular a soma do preço multiplicado pela quantidade de cada produto.

RF06 - Busca de produtos: Localizar produtos por código, nome ou categoria.

RF07 - Histórico: Registrar e consultar entradas e saídas com produto, tipo, quantidade, data e saldo resultante.

4.2 Regras de negócio

RN01 - Código único: Não permitir dois produtos com o mesmo código, e exigir código maior que zero.

RN02 - Saldo disponível: Não permitir uma saída superior à quantidade atual do produto.

RN03 - Valores válidos: Exigir preço maior que zero e impedir quantidade inicial ou estoque mínimo negativos.

RN04 - Movimentação válida: Exigir quantidade movimentada maior que zero e tipo igual a entrada ou saída.

RN05 - Produto existente: Permitir movimentações somente para produtos previamente cadastrados.

RN06 - Critério de alerta: Considerar crítico o produto cuja quantidade seja menor ou igual ao estoque mínimo.

Além das regras numeradas, nome e categoria são campos obrigatórios nas duas soluções.

5 Paradigma orientado a objetos com Java

Java foi escolhida por oferecer suporte direto a classes, objetos, encapsulamento, métodos e tipagem estática. A implementação utiliza Spring Boot para iniciar a aplicação web, Thymeleaf para apresentar os dados e uma separação entre modelo, serviço e controlador.

5.1 Organização da solução Java

Elemento	Responsabilidade
Produto	Representa os dados do produto e concentra operações de quantidade, cálculo de valor e verificação de estoque crítico.
Movimentacao	Representa cada entrada ou saída, incluindo produto, tipo, quantidade, data e saldo resultante.
ControleEstoque	Mantém as coleções, aplica regras de negócio, executa buscas, movimentações e cálculos.
EstoqueController	Recebe as ações da interface web, chama o serviço e fornece os resultados para a página.
index.html	Apresenta dashboard, cadastro, inventário, movimentações, histórico e alertas.

A quantidade permanece encapsulada no objeto Produto. O serviço solicita a adição ou a remoção por métodos específicos, evitando que a regra de saldo seja espalhada pela aplicação.

5.2 Estrutura resumida em Java

```
Produto
                        adicionarQuantidade()                          removerQuantidade()
              isEstoqueCritico()                                           getValorTotal()

Movimentacao
          produto          tipo          quantidade          data          saldoResultante

ControleEstoque
                         cadastrarProduto()                       registrarMovimentacao()
                             buscar()                          verificarEstoqueCritico()
  calcularValorEstoque()  listarHistorico()
```

6 Paradigma procedural com Python

Python foi escolhida por sua sintaxe direta e pela facilidade de organizar a aplicação em funções. Embora a linguagem também ofereça orientação a objetos, esta solução foi implementada propositalmente sem classes para evidenciar o paradigma procedural.

6.1 Organização da solução Python

Elemento	Responsabilidade
estoque	Lista de dicionários que representa os produtos cadastrados.
historico	Lista de dicionários que registra as entradas e saídas.
Funções de negócio	Validam e transformam os dados, como cadastrar_produto e registrar_movimentacao.

Elemento	Responsabilidade
Funções de consulta	Buscam produtos, verificam estoque crítico, calculam valor e retornam o histórico.
Funções de interface	Leem entradas, apresentam tabelas no terminal e controlam o menu.

6.2 Estrutura resumida em Python

```
estoque                                 =                                       []
historico                                =                                      []

buscar_por_codigo()                                                 cadastrar_produto()
registrar_movimentacao()                                          consultar_inventario()
buscar_produtos()                                               verificar_estoque_critico()
calcular_valor_total_estoque()  consultar_historico()
```

O fluxo parte da função main, apresenta o menu, recebe a opção, chama a função responsável, valida os dados, executa a operação, mostra o resultado e retorna ao menu. As listas são passadas como parâmetros, o que torna explícito quais dados cada procedimento consulta ou modifica.

7 Equivalência entre as implementações

Funcionalidade	Java OO	Python procedural
Cadastro	ControleEstoque.cadastrarProduto	cadastrar_produto
Entrada e saída	ControleEstoque.registrarMovimentacao	registrar_movimentacao
Inventário	ControleEstoque.listarProdutos	consultar_inventario
Busca	ControleEstoque.buscar	buscar_produtos
Estoque crítico	ControleEstoque.verificarEstoqueCritico	verificar_estoque_critico
Valor total	ControleEstoque.calcularValorEstoque	calcular_valor_total_estoque
Histórico	ControleEstoque.listarHistorico	consultar_historico

As soluções possuem interfaces diferentes: Java oferece uma página web e Python utiliza o terminal. Essa diferença não altera o núcleo funcional, pois as mesmas entradas válidas produzem atualizações equivalentes e as mesmas situações inválidas são rejeitadas.

8 Comparação técnica

Critério	Java orientado a objetos	Python procedural
Divisão do problema	Classes organizadas em modelo, serviço e controlador.	Funções agrupadas por cadastro, movimentação, consulta e interface.
Elemento principal	Objetos das classes Produto e Movimentacao.	Funções, listas e dicionários.
Estado e dados	Encapsulados nos objetos e coleções do serviço.	Mantidos em listas e dicionários passados às funções.
Regras	Métodos de Produto e ControleEstoque lançam exceções.	Funções validam os parâmetros e lançam ValueError.
Interface	Aplicação web com Spring Boot e Thymeleaf.	Menu textual executado no terminal.
Legibilidade	Responsabilidades explícitas, com mais arquivos e estrutura.	Fluxo direto e concentrado em poucas estruturas.
Manutenção	Favorece expansão por separação de responsabilidades.	Adequada ao escopo pequeno; exige disciplina à medida que cresce.
Vantagem	Encapsulamento, organização e extensibilidade.	Simplicidade, rapidez de escrita e leitura imediata.
Limitação	Maior configuração e quantidade de código.	Menor proteção estrutural dos dados em sistemas grandes.

9 Implementação e validação na N2

A implementação manteve o problema de mercado e ampliou o núcleo mínimo da N1 com busca e histórico, aplicados de forma equivalente. A interface web foi adotada somente na versão Java; essa decisão não modificou as regras compartilhadas.

Solução	Tecnologias	Execução	Validação
Java	Java 17, Spring Boot, Maven e Thymeleaf	Aplicação web em http://localhost:8080	Testes JUnit e demonstração funcional
Python	Python 3.10 ou superior	Aplicação de terminal com python main.py	12 testes unittest e demonstração funcional

9.1 Instruções resumidas

Java: abrir o terminal na pasta do projeto, executar mvn spring-boot:run e acessar http://localhost:8080. Para os testes, executar mvn test.

Python: abrir o terminal na pasta do projeto e executar python main.py. Para os testes, executar python -m unittest -v.

Os dados das duas aplicações permanecem em memória e são apagados quando a execução é encerrada. Essa limitação está registrada e não interfere na comparação dos paradigmas.

10 Conclusão

As duas implementações demonstram que o mesmo problema pode ser resolvido com paradigmas diferentes sem alterar seus requisitos centrais. Na versão Java, classes e objetos concentram dados e comportamentos, enquanto o serviço coordena as regras e o controlador liga a lógica à interface. Na versão Python, listas e dicionários armazenam o estado e funções executam as operações em uma sequência explícita.

Na prática, Java exigiu uma estrutura maior, mas tornou as responsabilidades mais visíveis e favoreceu a evolução da solução. Python permitiu uma implementação mais curta e direta, adequada ao porte acadêmico do sistema. Portanto, ambos os paradigmas se mostraram capazes de resolver o problema; a principal diferença está na organização do código, no controle do estado e na forma de manutenção.

11 Referências

- SENAI FATESG. Atividade Avaliativa N1: Um problema, dois paradigmas. Goiânia, 2026.
- SENAI FATESG. Atividade Avaliativa N2: Implementação, demonstração e documentação atualizada. Goiânia, 2026.
- SEBESTA, Robert W. Conceitos de Linguagens de Programação. 11. ed. Porto Alegre: Bookman, 2018.
- DEITEL, Paul; DEITEL, Harvey. Java: Como Programar. 10. ed. São Paulo: Pearson, 2016.
- PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. Disponível em: https://docs.python.org/3/. Acesso em: 2 out. 2026.
- SPRING. Spring Boot Reference Documentation. Disponível em: https://docs.spring.io/spring-boot/. Acesso em: 2 out. 2026.
- THYMELEAF. Thymeleaf Documentation. Disponível em: https://www.thymeleaf.org/documentation.html. Acesso em: 2 out. 2026.